Absolutely. For the **Insurance Testing Sandbox** we created, I would define the validation rules at the **field level** and the **business-rule level**. This will give you a good basis for writing both manual test cases and Playwright automation.

1. Customer Registration – Data Validation Rules

Field	Validation Rule	Example Valid	Example Invalid
First Name	Mandatory	`John`	Blank
First Name	Alphabetic characters and spaces only	`John David`	`John123`
First Name	Maximum length	50 chars	>50 chars
Last Name	Mandatory	`Smith`	Blank
Last Name	Alphabetic characters and spaces only	`Smith-Jones`*	`Smith123`
Date of Birth	Mandatory	`15/06/1985`	Blank
Date of Birth	Must be a valid date	`15/06/1985`	`31/02/1985`
Date of Birth	Customer must be at least 18	25 years	16 years
Date of Birth	Future date not allowed	—	`01/01/2030`
Email	Mandatory	`john@test.com`	Blank
Email	Valid email format	`john.smith@test.com`	`john@test`
Phone	Mandatory	`4165550189`	Blank
Phone	Numeric	`4165550189`	`416ABC0189`
Phone	Exactly 10 digits	`4165550189`	`416555018`
Address	Mandatory	`25 Main Street`	Blank
Address	Maximum length	≤250 chars	>250 chars

* Allow hyphens only if that is part of the agreed business requirement.

2. Insurance Quote – Data Validation Rules

Input validation

Field	Validation Rule	Valid	Invalid
Insurance Type	Mandatory	Auto	Blank
Age	Mandatory	35	Blank
Age	Minimum 18	18	17
Age	Maximum 85	85	86
Age	Numeric only	35	`ABC`
Vehicle/Asset Value	Mandatory	25000	Blank
Vehicle/Asset Value	Must be > 0	1	0
Vehicle/Asset Value	Numeric	25000	`ABC`
Claims	Mandatory	0	Blank
Claims	Integer	2	`2.5`
Claims	Must be >= 0	0	-1
Location Risk	Mandatory	Low	Blank
Deductible	Numeric	1000	`ABC`
Deductible	Must be >= 0	500	-100

---

3. Risk Classification Rules

These are **business validation rules**, rather than simple field validations.

High Risk

Customer is classified as **High Risk** if **any** of these conditions is true:

```
Claims >= 3
OR
Location Risk = High
OR
Age < 25
```

Medium Risk

If the customer is not High Risk, classify as **Medium Risk** when:

```
Claims >= 1
OR
Location Risk = Medium
OR
Age >= 65
```

Low Risk

If neither High nor Medium conditions apply:

```
Risk = Low
```

Important boundary tests

Age	Claims	Location	Expected Risk
24	0	Low	High
25	0	Low	Low
64	0	Low	Low
65	0	Low	Medium
85	0	Low	Medium
30	0	High	High
30	3	Low	High
30	1	Low	Medium
30	0	Medium	Medium

These are excellent **boundary-value and decision-table tests** for Playwright.

4. Premium Calculation Validation Rules

For our sandbox, the premium calculation is:

Base premium

```
Base Premium = $1,000
```

Risk surcharge

Risk	Surcharge
Low	$0
Medium	$200
High	$500

Optional coverage

Coverage	Additional Premium
Collision	$200
Comprehensive	$300
Roadside Assistance	$100

No-claims discount

```
Claims = 0
→ Discount = $100
```

Otherwise:

```
Claims > 0
→ Discount = $0
```

Premium calculation

```
Subtotal =
Base Premium
+ Risk Surcharge
```

```
+ Optional Coverage
- Discount
```

Then:

```
Tax = Subtotal × 13%
```

And:

```
Final Premium = Subtotal + Tax
```

Rounded to **2 decimal places**.

Example

Customer:

```
Base Premium       $1,000
Risk               Medium
Risk surcharge     $200
Collision          $200
Comprehensive      $300
Roadside           $100
No claims discount -$100
--------------------------------
Subtotal            $1,700
Tax 13%               $221
--------------------------------
Final Premium       $1,921
```

---

5. Premium Calculation Test Rules

You should specifically test:

Minimum

```
Low Risk
0 claims
No optional coverage
```

Expected:

```
1000 - 100 = 900
Tax = 117
Final = 1017
```

High Risk

```
High Risk
3 claims
No optional coverage
```

Expected:

```
1000 + 500 = 1500
Tax = 195
Final = 1695
```

All optional coverages

```
Collision       200
Comprehensive   300
Roadside        100
```

Additional coverage:

```
600
```

Rounding

Test values that result in more than two decimal places before rounding.

Expected:

```
Final Premium = 2 decimal places
```

---

6. Payment – Data Validation Rules

Field	Rule
Cardholder Name	Mandatory
Cardholder Name	Alphabetic characters/spaces
Card Number	Mandatory
Card Number	Exactly 16 digits
Card Number	Numeric only

Field	Rule
Expiry	Mandatory
Expiry	Format MM/YY
Expiry Month	01–12
Expiry Date	Must not be expired
CVV	Mandatory
CVV	Exactly 3 digits
CVV	Numeric only

Business rule

For our sandbox:

```
Card number ending in 0000
→ Payment Declined
```

Any valid 16-digit card not ending in `0000`:

```
→ Payment Successful
```

Important tests

```
4111111111111111 → Success
4111111111110000 → Declined
411111111111111  → Invalid length
41111111111111A → Invalid characters
```

---

7. Policy Issuance – Data Validation Rules

A policy can be issued only when:

```
Quote exists
AND
Quote is valid
AND
Payment is successful
```

Policy number

Must:

- Be generated automatically
- Be unique
- Not be blank
- Follow the required format

Example:

```
POL10001
POL10002
POL10003
```

Policy status

On successful issuance:

```
ACTIVE
```

Effective date

```
Effective Date <= Expiry Date
```

Duplicate policy prevention

The same successful transaction must **not create two policies**.

This is an important test:

> Double-click Pay → only one policy should be generated.

---

8. Policy Modification – Validation Rules

A policy can be modified only when:

```
Policy Status = ACTIVE
```

Coverage modification

New coverage must:

- Not be blank
- Be a valid coverage option
- Trigger premium recalculation

Example:

```
Basic → Comprehensive
```

should result in:

```
New Premium != Old Premium
```

Cancelled policy

```
CANCELLED
→ Modification not allowed
```

Endorsement

Every modification should generate an endorsement/reference number.

Example:

```
END10001
```

---

9. Policy Cancellation – Validation Rules

Cancellation should be allowed only for:

```
ACTIVE policy
```

Should not be allowed for:

```
Already CANCELLED
Expired policy
Non-existent policy
```

Refund calculation

Our sandbox uses:

```
Refund = Premium × 80%
```

Example:

```
Premium = $1,000

Refund = $800
```

The cancellation result should contain:

- Policy number
- Cancellation status
- Cancellation date
- Refund amount

---

10. Claims – Data Validation Rules

Field	Rule
Policy Number	Mandatory
Policy Number	Must exist
Policy Number	Must belong to an active policy
Loss Date	Mandatory
Loss Date	Valid date
Loss Date	Cannot be future date
Claim Type	Mandatory
Claim Amount	Mandatory
Claim Amount	Numeric
Claim Amount	> 0
Claim Description	Mandatory

Claim amount business rule

Our sandbox currently uses:

```
Maximum Claim Amount = Policy Premium × 5
```

Therefore:

```
Claim Amount <= Policy Premium × 5
```

is valid.

Anything greater should be rejected.

---

11. Claim Duplicate Validation

The system should prevent duplicate claims for the same event.

For example:

```
Policy = POL10001
Loss Date = 01/09/2026
Loss Type = Accident
```

Submitting the same claim again should ideally produce:

```
Duplicate claim
```

This is an important business rule to add if you're using the sandbox for realistic insurance testing.

---

12. Policy Search – Validation Rules

Input	Rule
Policy Number	Mandatory
Policy Number	Valid format
Policy Number	Existing policy
Policy Number	Case handling defined

Input	Rule
Invalid policy	Appropriate error
Blank search	Validation message

Example:

```
POL10001 → Policy found
POL99999 → Policy not found
ABC123 → Invalid policy number
Blank → Policy number required
```

---

13. Cross-Functional Data Validation

These are especially important for **end-to-end testing**.

Customer → Quote

```
Customer Age = 24
```

Quote should use:

```
Age = 24
Risk = High
```

Quote → Payment

The premium displayed during payment must equal:

```
Calculated Quote Premium
```

Payment → Policy

Successful payment:

```
Payment SUCCESS
      ↓
Policy ACTIVE
```

Failed payment:

```
Payment FAILED
      ↓
No policy should be issued
```

Policy → Claim

```
ACTIVE policy
      ↓
Claim allowed
```

But:

```
CANCELLED policy
      ↓
Claim rejected
```

Policy → Modification

```
ACTIVE
→ Modification allowed
```

```
CANCELLED
→ Modification rejected
```

---

14. High-Priority Validation Matrix

For your **risk-based insurance testing**, I would prioritize these:

Priority	Functionality	Most Important Validation
P1	Premium	Correct calculation
P1	Payment	Correct success/failure
P1	Policy issuance	Policy only after successful payment
P1	Claims	Correct eligibility and claim amount
P1	Cancellation	Correct refund
P2	Risk	Correct risk classification

Priority	Functionality	Most Important Validation
P2	Quote	Correct quote generation
P2	Policy modification	Correct premium recalculation
P2	Renewal	Correct eligibility
P3	Customer	Field validation
P3	Search	Correct policy retrieval
P4	UI	Formatting/layout

For Playwright automation

These rules translate very nicely into automated tests. For example:

TypeScript


```
ge }) => {
    await page.locator('#locationRisk').selectOption('Low');

    await page.getByRole('button', { name: 'Calculate Quote' }).click();

    await expect(page.locator('#riskResult')).toHaveText('High');
});
```

And the **premium calculation rules** should ideally also be tested with **data-driven tests**, so you can test Low/Medium/High risk, claims, coverage combinations, and boundary values without duplicating test code.